Solution: Remove K Digits

Problem Statement

Given a non-negative integer represented as a string `num` and an integer `k`, delete `k` digits from `num` to obtain the smallest possible integer. Return this minimum possible integer as a string.

Examples

1. - Input: `num = "1432219", k = 3`
   - Output: `"1219"`
   - Explanation: The digits removed are 4, 3, and 2 forming the new number 1219 which is the smallest.
2. - Input: `num = "10200", k = 1`
   - Output: `"200"`
   - Explanation: Removing the leading 1 forms the smallest number 200.
3. - Input: `num = "1901042", k = 4`
   - Output: `"2"`
   - Explanation: Removing 1, 9, 1, and 4 forms the number 2 which is the smallest possible.

Solution

The idea is to keep removing the peak digit from the number `num`. The peak digit in a number is a digit after which the next digit is smaller. By doing so, we are always trying to minimize the leftmost digit which will give us the smallest possible number.

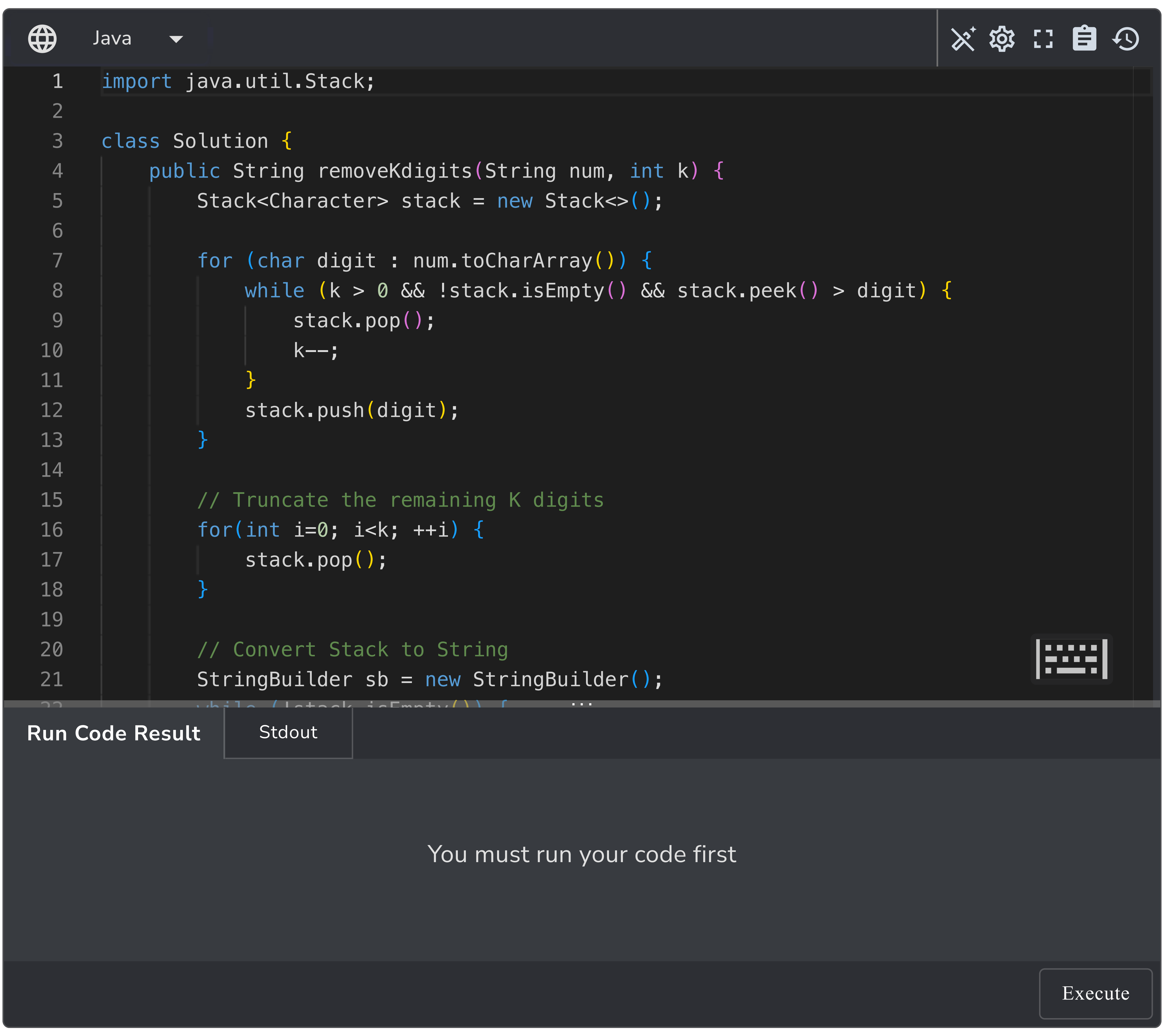
We can use a monotonically increasing stack to keep track of the decreasing peak digits.

Algorithm Walkthrough

1. Initialize an empty stack.
2. Iterate over the digits in `num`.
3. For each digit:
   - While `k` is greater than 0, the stack is not empty and the current digit is smaller than the top digit on the stack, pop the top digit from the stack and decrease `k` by 1.
   - Push the current digit onto the stack.
4. If `k` is still greater than 0, pop `k` digits from the stack.
5. Build the result string from the digits in the stack. Remove the leading zeros.
6. If the result string is empty, return "0". Otherwise, return the result string.

Code

Here is how we can implement this algorithm:



```java
import java.util.Stack;

class Solution {
    public String removeKdigits(String num, int k) {
        Stack<Character> stack = new Stack<>();

        for (char digit : num.toCharArray()) {
            while (k > 0 && !stack.isEmpty() && stack.peek() > digit) {
                stack.pop();
                k--;
            }
            stack.push(digit);
        }

        // Truncate the remaining K digits
        for(int i=0; i<k; ++i) {
            stack.pop();
        }

        // Convert Stack to String
        StringBuilder sb = new StringBuilder();
```

You must run your code first

Time and Space Complexity

- Time: $O(N)$ where N is the number of digits in input.
- Space: $O(N)$ to store stack.

← Previous
Remove K Digits (Hard)
Next →
Introduction To Hashing
☐ Mark as Completed